Listat & Tuplet në Python

Ushtrime — Niveli II

Programim në Python

October 1, 2025

Përmbajtja

1. Ushtrimet 1–7
2. Ushtrimet 1–4
3. Ushtrimet 5–7
4. Ushtrimet 8–10
5. Ushtrimet 11–13

Përmbajtja

1. Ushtrimet 1–7
2. Ushtrimet 1–4
3. Ushtrimet 5–7
4. Ushtrimet 8–10
5. Ushtrimet 11–13

Përmbajtja

1. Ushtrimet 1–7
2. Ushtrimet 1–4
3. Ushtrimet 5–7
4. Ushtrimet 8–10
5. Ushtrimet 11–13

1) Total Sales — Pyetja

Dizajnoni program që kërkon shitjet e dyqanit për çdo ditë të javës. Shumat ruhen në një listë. Përdorni një cikël për të llogaritur totalin javor dhe shfaqeni rezultatin.

1) Total Sales — Zgjidhja bazë

```python
def main():
    days = ['Sunday','Monday','Tuesday','Wednesday','Thursday','Friday','Saturday']
    sales = [0.0] * 7

    for i, day in enumerate(days):
        sales[i] = float(input(f'Enter the sales for {day}: '))

    total = 0.0
    for amount in sales:
        total += amount

    print(f'Total sales for the week: ${total:,.2f}')
```

Shpjegim. Rujmë vlerat në listë dhe pastaj i akumulojmë me një cikël.

1) Total Sales — Variant (shkurt me sum)

```
days = ['Sunday','Monday','Tuesday','Wednesday','Thursday','Friday','
    Saturday']
sales = [float(input(f'Enter the sales for {d}: ')) for d in days]
print(f'Total sales for the week: ${sum(sales):,.2f}')
```

Ide. `sum(lista)` i mbledh direkt elementët; list comprehension e bën futjen kompakte.

2) Lottery Number Generator — Pyetja

Dizajnoni program që gjeneron një numër lotarie me 7 shifra: krijoni një listë me 7 numra të rastësishëm në intervalin **0..9** dhe pastaj shfaqni listën.

2) Lottery Number Generator — Zgjidhja

```python
import random

def main():
    digits = [random.randint(0, 9) for _ in range(7)]
    # Shfaqje si numër i bashkuar:
    print('Lottery:', ''.join(str(d) for d in digits))
    # Ose si listë:
    print('List form:', digits)
```

Shpjegim. `randint(0,9)` përfshin skajet; `join` bashkon shifrat pa presje.

3) Rainfall Statistics — Pyetja

Lejoni përdoruesin të fusë reshjet mujore për 12 muaj (në listë). Llogaritni dhe shfaqni: **totalin vjetor**, **mesataren mujore**, **muajin me sasinë më të lartë**, **muajin me sasinë më të ulët**.

3) Rainfall Statistics — Zgjidhja

```python
def main():
    months = ['January','February','March','April','May','June',
              'July','August','September','October','November','December']
    rain = [0.0] * 12

    for i, m in enumerate(months):
        rain[i] = float(input(f'Enter the rainfall for {m}: '))

    total = sum(rain)
    avg = total / len(rain)
    hi = max(rain); lo = min(rain)
    i_hi = rain.index(hi); i_lo = rain.index(lo)

    print(f'Total rainfall: {total:.2f}')
    print(f'Average monthly: {avg:.2f}')
    print(f'Highest: {months[i_hi]} ({hi:.2f})')
    print(f'Lowest : {months[i_lo]} ({lo:.2f})')
```

4) Number Analysis — Pyetja

Kërkoni 20 numra nga përdoruesi, ruani në listë dhe shfaqni: **më të voglin**, **më të madhin**, **totalin**, **mesataren**.

4) Number Analysis — Zgjidhja

```python
def main():
    nums = []
    for i in range(20):
        nums.append(float(input(f'Enter number {i+1}/20: ')))

    low, high = min(nums), max(nums)
    total = sum(nums)
    avg = total / len(nums)

    print(f'Low: {low}')
    print(f'High: {high}')
    print(f'Total: {total:,.2f}')
    print(f'Average: {avg:,.2f}')
```

4) Variant (me `try/except` dhe rifutje)

```python
def read_float(prompt: str) -> float:
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print('Ju lutem jepni një numër të vlefshëm!')

nums = [read_float(f'Nr {i+1}/20: ') for i in range(20)]
print(min(nums), max(nums), sum(nums), sum(nums)/len(nums))
```

Ide. Rrit robustësinë ndaj inputeve jo-numerike.

4) Variant — Shpjegim

Ky variant përdor funksionin `read_float` për të siguruar që inputi i përdoruesit të jetë gjithmonë numër real.

- **Rifutje automatike:** Nëse përdoruesi gabon, kërkohet sërish inputi derisa të jepet vlerë e vlefshme.
- **List comprehension:** Të gjitha vlerat mblidhen në një listë me një rresht të shkurtër.
- **Përfitimi:** Programi nuk ndërpritet nga gabimet e inputit dhe është më i lehtë për t'u përdorur.

Kjo qasje është më e sigurt dhe më e përshtatshme për përdoruesit fillestarë.

Përmbajtja

1. Ushtrimet 1–7
2. Ushtrimet 1–4
3. Ushtrimet 5–7
4. Ushtrimet 8–10
5. Ushtrimet 11–13

5) Charge Account Validation — Pyetja

Lexoni `charge_accounts.txt` në një listë. Kërkoni një numër llogarie nga përdoruesi dhe kontrolloni nëse gjendet në listë. Shfaqni mesazh për **valid**/**invalid**.

5) Charge Account Validation — Zgjidhja

```python
def main():
    try:
        with open('charge_accounts.txt', 'r', encoding='utf-8') as f:
            accounts = [line.strip() for line in f if line.strip()]
    except IOError:
        print('Skedari nuk u gjet.'); return

    test = input('Enter the account number to be validated: ').strip()
    if test in accounts:
        print(f'Account number {test} is valid.')
    else:
        print(f'Account number {test} is not valid.')
```

6) Larger Than n — Pyetja

Shkruani një funksion që pranon: (1) një listë & (2) një numër `n`. Funksioni *shfaq* të gjitha numrat në listë që janë **më të mëdhenj se** `n`.

6) Larger Than n — Zgjidhja (sipas kërkesës)

```python
def display_larger_than_n_list(n, n_list):
    larger = []
    for value in n_list:
        if value > n:
            larger.append(value)
    print(larger)

# Shembull përdorimi:
number = 5
number_list = [1,2,3,4,5,6,7,8,9,10]
display_larger_than_n_list(number, number_list)
```

Shpjegim. Ndërtojmë një listë filtruar dhe e shfaqim.

6) Variant (kthim vlere & list comprehension)

```
def larger_than(n, seq):
    return [x for x in seq if x > n]

print(larger_than(5, [1,2,3,4,5,6,7,8,9,10]))
```

Ide. Kthimi i listës së filtruar e bën funksionin të ripërdorshëm në pjesë të tjera të kodit.

7) Driver's License Exam — Pyetja

Egzamin me 20 pyetje *multiple-choice*. Ruani **përgjigjet e sakta** në listë. Lexoni **përgjigjet e studentit** nga skedari (një në rresht) në listë tjetër. Shfaqni **PASS/FAIL** (**kalon** me **15/20** ose më shumë), **numrin e saktave**, **numrin e gabuara**, dhe **indekset e pyetjeve të gabuara**.

7) Driver's License Exam — Zgjidhja I

```python
CORRECT = ['A','C','A','A','D','B','C','A','C','B',
           'A','D','C','A','D','C','B','B','D','A']

def read_answers(path):
    with open(path, 'r', encoding='utf-8') as f:
        return [line.strip().upper() for line in f if line.strip()]

def grade(correct, student):
    n = min(len(correct), len(student))
    wrong = []
    right = 0
    for i in range(n):
        if student[i] == correct[i]:
            right += 1
        else:
            wrong.append(i+1) # pyetje 1-based
```

7) Driver's License Exam — Zgjidhja II

```
17
18      if len(student) < len(correct):
19          wrong.extend(range(len(student)+1, len(correct)+1))
20
21      passed = right >= 15
22      return passed, right, len(correct) - right, wrong
23
24  def main():
25      student = read_answers('student_answers.txt')
26      passed, right, wrong_cnt, wrong_indices = grade(CORRECT, student)
27
28      print('Result:', 'PASS' if passed else 'FAIL')
29      print('Correct:', right)
30      print('Incorrect:', wrong_cnt)
31      if wrong_indices:
32          print('Wrong questions:', wrong_indices)
```

7) Driver's License Exam — Zgjidhja III

Shpjegim. Krahasim element-më-element; mbajmë lista me indekset e gabuara. Kalon kur `right >= 15`.

Përmbajtja

1. Ushtrimet 1–7
2. Ushtrimet 1–4
3. Ushtrimet 5–7
4. Ushtrimet 8–10
5. Ushtrimet 11–13

8) Name Search — Pyetja

Lexoni përmbajtjen e `BoyNames.txt` dhe `GirlNames.txt` në dy lista të ndara. Përdoruesi fut një emër djali, një emër vajze, ose të dy. Shfaqni nëse secili ishte ndër 200 emrat më të popullarizuar (2000–2009).

8) Name Search — Zgjidhja I

```
def main():
    # Setup variables
    boy = ''
    girl = ''

    try:
        boy_input = open('BoyNames.txt', 'r')
        popular_boys = boy_input.readlines()

        for i in range(len(popular_boys)):
            popular_boys[i] = popular_boys[i].rstrip('\n')

        girl_input = open('GirlNames.txt', 'r')

        popular_girls = girl_input.readlines()

```

8) Name Search — Zgjidhja II

```
17        for i in range(len(popular_girls)):
18            popular_girls[i] = popular_girls[i].rstrip('\n')
19
20        # Obtain user input.
21        boy = input("Enter a boy's name, or N if you do not wish to enter a boy's
              name: ")
22        girl = input("Enter a girl's name, or N if you do not wish to enter a girl's
              name: ")
23
24        if boy=='N':
25            print("You chose not to enter a boy's name.")
26        elif boy in popular_boys:
27            print(f"{boy} is one of the most popular boy's names.")
28        else:
29            print(f"{boy} is not one of the most popular boy's names.")
30
31        if girl=='N':
```

8) Name Search — Zgjidhja III

```
32            print("You chose not to enter a girl's name.")
33        elif girl in popular_girls:
34            print(f"{girl} is one of the most popular girl's names.")
35        else:
36            print(f"{girl} is not one of the most popular girl's names.")
37
38    except IOError:
39        print("The file could not be found.")
40    except IndexError:
41        print("There was an indexing error.")
42    except:
43        print("An error occurred.")
```

8) Variant Name Search — Zgjidhja I

```python
def main():
    try:
        with open('BoyNames.txt', 'r', encoding='utf-8') as f:
            popular_boys = [line.strip() for line in f if line.strip()]
        with open('GirlNames.txt', 'r', encoding='utf-8') as f:
            popular_girls = [line.strip() for line in f if line.strip()]
    except IOError:
        print('Skedarët nuk u gjetën.'); return

    boy = input("Boy's name (ose N për të anashkaluar): ").strip()
    girl = input("Girl's name (ose N për të anashkaluar): ").strip()

    if boy.upper() == 'N':
        print("S'keni futur emër djali.")
    else:
```

8) Variant Name Search — Zgjidhja II

```
        print(f"{boy} është", "në listë." if boy in popular_boys else "JASHTË listës.")

    if girl.upper() == 'N':
        print("S'keni futur emër vajze.")
    else:
        print(f"{girl} është", "në listë." if girl in popular_girls else "JASHTË listës.")
```

9) Population Data — Pyetja

Lexoni `USPopulation.txt` (popullsia në mijëra, 1950–1990). Shfaqni: **mesataren e ndryshimit vjetor**, **vitin me rritjen më të madhe**, **vitin me rritjen më të vogël**.

9) Population Data — Zgjidhja bazë I

```
def main():

    # Setup variables
    yearly_change = []
    change = 0.0
    total_change = 0.0
    average_change = 0.0
    greatest_increase = 0.0
    smallest_increase = 0.0
    greatest_year = 0
    smallest_year = 0

    BASE_YEAR = 1950

    try:
        input_file = open('USPopulation.txt', 'r')
```

9) Population Data — Zgjidhja bazë II

```
17
18      yearly_population = input_file.readlines()
19
20      for i in range(len(yearly_population)):
21          yearly_population[i] = float(yearly_population[i])
22
23      for i in range(1, len(yearly_population)):
24          change = yearly_population[i] - yearly_population[i-1]
25          yearly_change.append(change)
26
27          if i==1:
28              greatest_increase = change
29              smallest_increase = change
30              greatest_year = 1
31              smallest_year = 1
32
33          else:
```

9) Population Data — Zgjidhja bazë III

```
            if change > greatest_increase:
                greatest_increase = change
                greatest_year = i

            elif change < smallest_increase:
                smallest_increase = change
                smallest_year = i

    total_change = float(sum(yearly_change))
    average_change = total_change / len(yearly_change)

    print(f'The average annual change in population '
          f'during the time period is {average_change:,.2f}')
    print(f'The year with the greatest increase in population was '
          f'{BASE_YEAR + greatest_year}')
    print(f'The year with the smallest increase in population was '
          f'{BASE_YEAR + smallest_year}')
```

9) Population Data — Zgjidhja bazë IV

```
51
52      except IOError:
53          print('The file could not be found.')
54      except IndexError:
55          print('There was an indexing error.')
56      except:
57          print('An error occurred.')
```

9) Variant Population Data — Zgjidhja I

```python
def main():
    BASE_YEAR = 1950
    try:
        with open('USPopulation.txt', 'r', encoding='utf-8') as f:
            pops = [int(line.strip()) for line in f if line.strip()]
    except IOError:
        print('Skedari nuk u gjet.'); return

    changes = [pops[i] - pops[i-1] for i in range(1, len(pops))]
    avg_change = sum(changes) / len(changes)

    # indeks 0 i changes i referohet vitit BASE_YEAR+1, etj.
    idx_max = max(range(len(changes)), key=changes.__getitem__)
    idx_min = min(range(len(changes)), key=changes.__getitem__)
    year_max = BASE_YEAR + 1 + idx_max
```

9) Variant Population Data — Zgjidhja II

```
16    year_min = BASE_YEAR + 1 + idx_min
17
18    print(f'Mesatarja e ndryshimit vjetor: {avg_change:,.2f}')
19    print(f'Rritja më e madhe: viti {year_max}')
20    print(f'Rritja më e vogël: viti {year_min}')
```

10) World Series Champions — Pyetja

Lexoni `WorldSeriesWinners.txt` (1903–2009; pa 1904, 1994). Përdoruesi jep një ekip; shfaqni sa herë ka fituar në atë periudhë.
Shembull nga skedari:

```
Boston Americans
New York Giants
Chicago White Sox
Chicago Cubs
Chicago Cubs
```

10) World Series Champions — Zgjidhja I

```
def main():

    team = ''
    win_count = 0

    try:
        input_file = open('WorldSeriesWinners.txt', 'r')

        winners = input_file.readlines()

        for i in range(len(winners)):
            winners[i] = winners[i].rstrip('\n')

        team = input('Enter the name of a team: ')

        if team not in winners:
```

10) World Series Champions — Zgjidhja II

```
            print(f'The {team} never won the world series.')

        else:
            for item in winners:
                if item == team:
                    win_count += 1
            print(f'The {team} won the world series '
                  f'{win_count} times between 1903 and 2009.')

    except IOError:
        print('The file could not be found.')
    except IndexError:
        print('There was an indexing error.')
    except:
        print('An error occurred.')
```

10) Variant World Series Champions — Zgjidhja

```python
def main():
    try:
        with open('WorldSeriesWinners.txt', 'r', encoding='utf-8') as f:
            winners = [line.strip() for line in f if line.strip()]
    except IOError:
        print('Skedari nuk u gjet.'); return

    team = input('Enter team name: ').strip()
    wins = sum(1 for w in winners if w == team)

    if wins == 0:
        print(f'The {team} never won the World Series (1903-2009).')
    else:
        print(f'The {team} won the World Series {wins} time(s) (1903-2009).'
            )
```

Përmbajtja

1. Ushtrimet 1–7
2. Ushtrimet 1–4
3. Ushtrimet 5–7
4. Ushtrimet 8–10
5. Ushtrimet 11–13

11) Prime Number Generation — Pyetja

Përdoruesi jep një $n > 1$. Krijoni listën `[2..n]` dhe për çdo element shfaqni nëse është **prime** apo **composite**. (Sugjerim: funksion ndarës `is_prime`.)

11) Prime Number Generation — Zgjidhja

```python
def prime_or_composite(n):
    has_divisor = False

    for i in range(2, n):
        if n % i == 0:
            has_divisor = True

    if has_divisor:
        print(f'{n} is composite.')
    else:
        print(f'{n} is prime.')
```

11) Prime Number Generation — Zgjidhja

```python
def main():
    user_num = int(input('Enter an integer greater than 1: '))

    numbers = []

    for count in range(2, user_num + 1):
        numbers.append(count)

    for i in range(len(numbers)):
        prime_or_composite(numbers[i])
```

11) Variant Prime Number Generation — Zgjidhja (sqrt kontroll)

```python
import math

def is_prime(k: int) -> bool:
    if k < 2: return False
    if k in (2,3): return True
    if k % 2 == 0: return False
    limit = int(math.isqrt(k))
    for d in range(3, limit+1, 2):
        if k % d == 0:
            return False
    return True

def main():
    n = int(input('Enter an integer > 1: '))
    for x in range(2, n+1):
        print(f'{x} is', 'prime.' if is_prime(x) else 'composite.')
```

13) Magic 8 Ball — Pyetja

Lexoni përgjigjet nga `8_ball_responses.txt` (12 rreshta). Pyesni përdoruesin dhe shfaqni një përgjigje të rastësishme. Përsëriteni derisa përdoruesi të zgjedhë të dalë. Skedari përmban:

```
Yes, of course!
Without a doubt, yes.
You can count on it.
For sure!
Ask me later.
I'm not sure.
I can't tell you right now.
I'll tell you after my nap.
No way!
I don't think so.
Without a doubt, no.
The answer is clearly NO.
```

13) Magic 8 Ball — Zgjidhja (me `random.choice`)

```
import random

def main():
    response_file = open('8_ball_responses.txt', 'r')

    responses = response_file.readlines()

    response_file.close()

    for i in range(len(responses)):
        responses[i] = responses[i].rstrip('\n')

    question = input('Enter your question: ')

    print(responses[random.randint(0, len(responses))])
```

13) Variant Magic 8 Ball — Zgjidhja (me `random.choice`)

```python
import random

def main():
    with open('8_ball_responses.txt', 'r', encoding='utf-8') as f:
        responses = [line.strip() for line in f if line.strip()]

    while True:
        q = input('Pyetje (ose shtyp ENTER për të dalë): ').strip()
        if q == '':
            print('Bye!'); break
        print(random.choice(responses))
```

Shënim. Përdorim `random.choice(responses)` për të shmangur gabimin *off-by-one* te `randint`.

Faleminderit!

Pyetje?